

Detecting Bugs in Rust Compiler Fix Suggestions via Constraint-Violation-Guided Mutation

ZIXI LIU, Nanjing University, China

YANG FENG*, Nanjing University, China

JIALIANG JIANG, Nanjing University, China

BAOWEN XU, Nanjing University, China

Rust is a modern systems programming language that ensures memory safety through unique mechanisms, including ownership, borrowing, and lifetime annotations. These features prevent critical vulnerabilities but also impose strict constraints that many developers find difficult to understand. To mitigate this challenge, the Rust compiler, `rustc`, provides rich diagnostics and fix suggestions. However, recent studies reveal that diagnostic issues account for about 20% of all reported `rustc` bugs. Our analysis of `rustc`'s suggestion bugs fixed over the past three years shows that most of them originated from errors in Rust-specific core modules, such as the type checker and borrow checker, rather than from simple mistakes in the general diagnostic logic, like suggesting an incorrect variable name or mismatched parentheses. The impact of diagnostic issues, especially bugs in `rustc`'s fix suggestion, should not be underestimated, as they can mislead developers and reduce `rustc`'s usability, and in severe cases may even lead to `rustc` crashes. Existing testing tools, however, provide little support for systematically evaluating the correctness and reliability of these suggestions.

To address this gap, in this paper, we present SugBreaker, an automated testing framework specifically designed to validate `rustc`'s suggestions. We propose a constraint-violation-guided mutation approach that injects type-related, borrow-related, and lifetime-related errors into valid Rust programs to trigger compiler diagnostics and iteratively verify the correctness of suggested fixes. SugBreaker has already detected 12 bugs, and 11 of them have been confirmed or fixed; all of them are triggered by different `rustc` error messages. Compared with a series of `rustc` testing baseline tools, SugBreaker achieves broader coverage of `rustc`'s core checking modules and a higher suggestion trigger rate, which further confirms the effectiveness and efficiency of SugBreaker for testing `rustc`'s fix suggestions.

CCS Concepts: • **Software and its engineering** → Software maintenance tools.

Additional Key Words and Phrases: Rust, compiler, testing, mutation

ACM Reference Format:

Zixi Liu, Yang Feng, Jialiang Jiang, and Baowen Xu. 2026. Detecting Bugs in Rust Compiler Fix Suggestions via Constraint-Violation-Guided Mutation. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE033 (July 2026), 22 pages. <https://doi.org/10.1145/3797128>

1 Introduction

Rust has emerged as a modern systems programming language that emphasizes both performance and safety. Unlike traditional languages such as C and C++, Rust introduces unique mechanisms

*Yang Feng is the corresponding author.

Authors' Contact Information: Zixi Liu, Nanjing University, State Key Laboratory for Novel Software Technology, Nanjing, China, zxliu@smail.nju.edu.cn; Yang Feng, Nanjing University, State Key Laboratory for Novel Software Technology, Nanjing, China, fengyang@nju.edu.cn; Jialiang Jiang, Nanjing University, State Key Laboratory for Novel Software Technology, Nanjing, China, 522025330038@smail.nju.edu.cn; Baowen Xu, Nanjing University, State Key Laboratory for Novel Software Technology, Nanjing, China, bw Xu@nju.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE033

<https://doi.org/10.1145/3797128>

such as ownership, borrowing, and lifetime annotations to ensure memory safety without relying on a garbage collector [2, 11, 20]. These mechanisms enable precise control over resource allocation and deallocation, thus preventing vulnerabilities such as dangling pointers, data races, buffer overflows, and use-after-free bugs. Owing to its unique memory management mechanisms, Rust has been widely recognized as a promising alternative for building reliable and secure systems software [3, 5, 23, 27, 30, 38, 39].

The benefits of these strong safety guarantees, however, come with a cost. The ownership and lifetime system imposes strict constraints that could be difficult to understand, especially for developers with prior experience in C or C++. Many developers find the learning curve steep, since programs that appear correct at first glance may fail to compile due to subtle restrictions enforced by the borrow checker, lifetime management rules, and Rust's expressive static type system [8, 29, 47]. In this context, the error messages and fix suggestions provided by the Rust compiler, `rustc`, play a crucial role. As a modern compiler, `rustc` provides a wide range of diagnostics and fix suggestions to help developers understand and resolve program errors. The suggestions provided by `rustc` are not limited to general syntactic issues, such as unmatched parentheses, but also extend to deeper compilation stages tied to Rust's unique features, such as borrow and type checking. By providing such guidance, `rustc` makes it easier for developers to comprehend and fix errors in Rust programs.

However, a recent empirical study on `rustc` bugs reveals that diagnostic issues account for about 20% of all reported bugs [18]. Moreover, more than 95% of these diagnostic issues were identified either by the Rust development team or by Rust users, rather than by existing `rustc` testing tools. This observation suggests that current testing and verification approaches largely overlook the diagnostic module. In practice, the number of diagnostic issues in `rustc` may be even higher, with many remaining undiscovered. Building upon the existing study, we further conduct a statistical analysis focusing specifically on `rustc`'s fix suggestion bugs, as presented in Section 2. The results show that these suggestion bugs are not primarily concentrated in error-reporting components. Instead, they are distributed across Rust-specific compilation stages, such as type checking and borrow checking, where `rustc` enforces Rust's unique language mechanisms. As these findings suggest, diagnostic issues in `rustc` are not only widespread but also concentrated in Rust-specific core checking modules, which makes their consequences particularly significant. On the one hand, incorrect or misleading fix suggestions reduce the overall usability of Rust, potentially confusing developers and leading to misconceptions, particularly when they involve Rust-specific features. On the other hand, flaws in the diagnostic module can even cause `rustc` to crash and trigger internal compiler errors (ICEs) in severe cases. For instance, a recently reported crash bug [41] occurred during unnecessary transmute checking in the MIR transformation module. Instead of providing the intended suggestion, `rustc` encountered an error that led the compiler to crash.

Therefore, an automated testing approach specifically designed for Rust's unique language mechanisms is clearly needed. While existing testing approaches [28, 40, 43] mainly focus on exposing crashes and validating the correctness of compilation results, they provide little support for evaluating whether the fix suggestions are accurate or safe to apply. To address this gap, in this paper, we present `SugBreaker`, an automated testing framework specifically designed to evaluate `rustc`'s suggestions. Specifically, to effectively trigger suggestions from Rust's core checking modules, `SugBreaker` mutates valid Rust programs by injecting carefully crafted errors to violate Rust constraints. These mutations are designed to provoke `rustc` into producing corresponding errors and suggestions. Overall, we define three main categories of mutation rules: type-related, borrow-related, and lifetime-related, which together comprise a total of 12 mutation rules. Once a program is mutated, `SugBreaker` feeds it to `rustc` with relevant compiler flags. If a suggestion is produced, `SugBreaker` applies the fix and recompiles the program. Successfully compiled programs are sent to the seed set for further mutation. A suggestion bug is detected when the suggested

fix forms a cycle, when the target location cannot be identified, or when the fix fails to resolve the original error. Otherwise, if the fix still fails but produces a different error, the suggestion is marked as suspicious for manual review. This process is repeated iteratively, allowing SugBreaker to explore rustc's suggestions across many program variations.

To evaluate the effectiveness of SugBreaker, we apply SugBreaker on the latest version of rustc for detecting bugs in the fix suggestion. We have discovered 12 bugs, 11 of which have been confirmed or fixed. All of the bugs are triggered by different compiler errors, indicating diverse bug-triggering causes. Compared with existing rustc testing baselines, SugBreaker covers more Rust-specific checking modules, enabling it to expose core suggestion errors. In terms of efficiency, SugBreaker performs mutations faster than other rustc testing tools and program generators, and its automated validation outpaces tools targeting MIR optimizations such as Rustlantis. Finally, we present selected bug cases and discuss potential causes of suggestion bugs as well as directions for future research. In summary, the contributions of this work are as follows:

- **Approach.** We have proposed a constraint-violation-guided mutation approach for testing and evaluating rustc's diagnostic suggestions. Our approach mutates valid Rust programs by injecting errors that violate key Rust constraints, effectively triggering suggestions from core checking modules and enabling systematic exploration of compiler behavior.
- **Tool.** We have implemented a testing framework, namely SugBreaker, which implements constraint-violation-guided mutation rules, including type-related, borrow-related, and lifetime-related rules. SugBreaker automatically applies rustc's suggested fixes to mutated programs, re-compiles to detect various types of suggestion bugs.
- **Study.** We have conducted extensive experiments to evaluate the effectiveness and efficiency of SugBreaker. The results show that 11 bugs detected by SugBreaker have been confirmed or fixed by the Rust development team. Moreover, SugBreaker outperforms baselines in rustc's suggestion triggering rate, rustc coverage, and mutation generation speed.

2 Motivation

While Rust's advanced memory-safety mechanisms provide strong guarantees, they also make understanding compiler diagnostics and fix suggestions challenging for developers. In particular, misleading or incorrect suggestions from rustc can be confusing and difficult to interpret, especially when they involve Rust-specific features such as lifetimes.

A motivating example. Figure 1 presents a real bug report from Rust's GitHub repository. In this case, the function `f` attempts to return a value of type `S<'_>` but uses a placeholder (`_`) as the return type, which is not permitted in function signatures. While rustc correctly reports this error, its suggested fix, i.e., replacing the placeholder with `S<'static>`, is incorrect. Suppose a developer new to Rust follows the suggestion and updates the code; the program still fails to compile with an error indicating that the returned value does not meet the required 'static bound. At this point, rustc offers no further suggestion, leaving the developer confused and potentially spending considerable time diagnosing the issue. In fact, 'static in Rust denotes a lifetime that spans the entire program, which can exceed the lifetime of the input parameter `s`. The correct fix is to use `S<'_>` to match the parameter's lifetime, allowing the function to compile successfully. This case illustrates how misleading fix suggestions can confuse developers and even

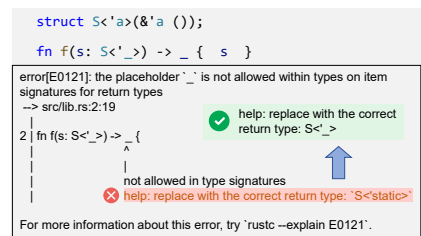


Fig. 1. A real bug example in rustc's fix suggestion (Issue 125488).

lead to incorrect code, particularly when Rust-specific features such as lifetimes are involved. These challenges underscore the need for automated approaches to detect and evaluate the bugs in `rustc` fix suggestion, ensuring that `rustc`'s feedback remains both accurate and reliable.

Table 1. Distribution of issues by number of affected modules.

Affected Modules	# Issues
1	152
2	26
3	12
4	3
5	2
6	2
7	1
Total	198

Table 2. Number of issues and corresponding descriptions for each affected module in `rustc`'s fix suggestion bugs.

Affected Module	# Issues	Description
Parser	64	<code>rustc_resolve</code> , <code>rustc_parse</code> , <code>rustc_ast_lowering</code>
Type Check	94	<code>rustc_hir_typeck</code> , <code>rustc_trait_selection</code> , <code>rustc_hir_analysis</code> , <code>rustc_ty_utils</code> , <code>rustc_infer</code>
Borrow Check	27	<code>rustc_mir_transform</code> , <code>rustc_passes</code> , <code>rustc_mir_build</code> , <code>rustc_borrowck</code>
Errors	39	<code>rustc_errors</code> , <code>rustc_lint</code>
Utils	23	<code>rustc_session</code> , <code>rustc_middle</code> , <code>rustc_expand</code> , <code>rustc_span</code> , <code>rustc_builtin_macros</code>

Empirical motivation. To understand the characteristics of `rustc`'s fix suggestion bugs, we build upon prior bug studies [18] and conduct a detailed analysis of real-world cases. We collected closed issues from the Rust GitHub repository reported between September 1, 2022, and September 1, 2025, with at least one of the following labels: *"D-invalid-suggestion"*, *"A-suggestion-diagnostics"*, or *"D-lack-of-suggestion"*. These labels, defined by the Rust team, directly reflect suggestion-related behavior. In total, we obtained 198 issues. For each issue, we examined the corresponding pull requests to identify the modules involved in the fix, excluding test files, and counted the proportion of issues affecting different numbers of modules, as shown in Table 1. About one quarter of the fixes involved more than one module, suggesting that many bugs require cross-module reasoning rather than simple local changes. We further categorized the affected modules along the compilation pipeline and recorded the frequency with which each was modified, as shown in Table 2. The results reveal that suggestion bugs often extend beyond error-reporting modules and are tightly linked to Rust-specific mechanisms such as type and borrow checking. This finding echoes the earlier motivating example and underscores that effective testing of `rustc`'s fix suggestions should target these internal checkers, which are fundamental to resolving such bugs.

Guided and motivated by these findings, we propose SugBreaker and design constraint-violation-guided mutation rules that target core `rustc` modules related to type checking, borrowing, lifetimes, and parsing. By injecting carefully crafted errors into these modules, SugBreaker triggers `rustc` to produce errors and suggestions, enabling systematic detection of suggestion bugs.

3 Approach

In this section, we present the design and implementation of SugBreaker, which provides a uniform framework for detecting `rustc`'s fix suggestion bugs. We begin by outlining the overall framework and its key design principles in Section 3.1, followed by a detailed description of our constraint-violation-guided mutation rules in Section 3.2. Finally, Section 3.3 describes the method for detecting `rustc` bugs in detail.

3.1 The Workflow of SugBreaker

The overall workflow of SugBreaker is depicted in Figure 2 and formally described in Algorithm 1. SugBreaker takes as input an initial seed set S of valid Rust programs, the total number of fuzzing iterations n , and a maximum number of suggestion-validation rounds m . At each iteration (Line 1),

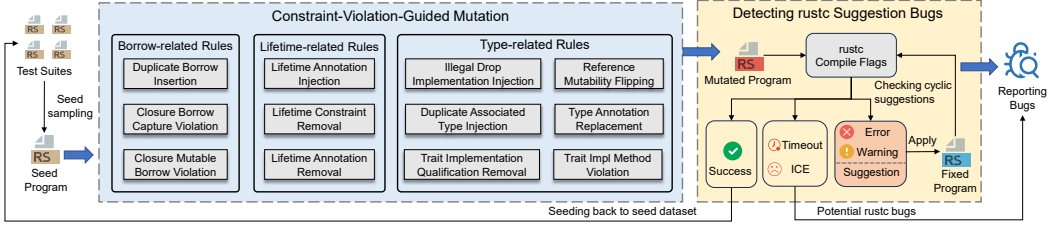


Fig. 2. The workflow of SugBreaker.

Algorithm 1: SugBreaker: Randomized Fuzzing of rustc Fix Suggestions

Input: Initial seed program set S , fuzz iterations n , max suggestion validation rounds m
Output: Detected suggestion bugs, performance issues, crashes

```

1  for  $t \leftarrow 1$  to  $n$  do
2       $p \leftarrow \text{RandomSelect}(S)$ ; // randomly select a seed program
3       $p_{\text{mut}} \leftarrow \text{RandomMutate}(p)$ ; // randomly apply a constraint-violation-guided mutation rule
4      if  $\text{CompilationSuccess}(p_{\text{mut}})$  then
5           $S \leftarrow S \cup \{p_{\text{mut}}\}$  continue; // add successfully compiled program to seed corpus
6      if  $\text{CompilationTimeout}(p_{\text{mut}})$  then
7          ReportBug("Performance",  $p_{\text{mut}}$ ) continue
8      if  $\text{CompilationCrash}(p_{\text{mut}})$  then
9          ReportBug("Crash",  $p_{\text{mut}}$ ) continue; // ICE, segmentation fault, or other crash
10      $p_{\text{cur}} \leftarrow p_{\text{mut}}$ 
11     for round  $\leftarrow 1$  to  $m$  do
12          $\text{suggestion} \leftarrow \text{CompileWithRustc}(p_{\text{cur}})$ 
13         if  $\text{suggestion} = \emptyset$  then
14             break
15          $p_{\text{cur}} \leftarrow \text{ApplySuggestion}(p_{\text{cur}}, \text{suggestion})$ 
16         if  $\text{CompilationSuccess}(p_{\text{cur}})$  then
17              $S \leftarrow S \cup \{p_{\text{cur}}\}$ ; // add fixed program to seed set
18             break
19     if not  $\text{CompilationSuccess}(p_{\text{cur}})$  then
20         ReportBug("Potential Suggestion Bug",  $p_{\text{mut}}$ )
    
```

SugBreaker randomly selects a seed program p from S (Line 2) and generates a mutated variant p_{mut} by applying a mutation randomly chosen from our set of type-related, borrow-related, and lifetime-related mutation rules specifically designed for Rust's language features (Line 3). The mutated program is then compiled with `rustc`. If the program compiles successfully, it is directly added to the seed set S for further fuzzing iterations (Lines 4–5). If timeouts or crashes are triggered, they are immediately reported as performance or crash bugs, respectively (Lines 6–9). Otherwise, SugBreaker enters a suggestion-validation phase, applying the compiler's fix suggestions iteratively. In each round, the current program p_{cur} (initially p_{mut}) is compiled, and any suggestion returned by `rustc` is applied to produce an updated program for the next round (Lines 10–15). This process continues until the program successfully compiles, no further suggestions are generated, or the iteration limit of m rounds is reached. After completing the suggestion-validation rounds, SugBreaker analyzes the final state of the program. If it successfully compiles, the fixed program is added to the seed set S to serve as input for future iterations, thereby continuously expanding the corpus of valid

programs and enabling further exploration of `rustc`'s behavior (Lines 16–18). If the program still fails to compile, it is recorded as a potential suggestion bug (Lines 19–20).

3.2 Constraint-Violation-Guided Mutation

To effectively trigger suggestions from `rustc`'s core modules, SugBreaker adopts a constraint-violation-guided mutation approach. Starting from valid Rust programs, it deliberately injects targeted violations to provoke compiler errors and generate corresponding fix suggestions. This section first outlines the design principles behind our mutations, followed by a detailed explanation of each mutation rule.

3.2.1 Design Principles. Rust implements several unique language mechanisms, such as borrow checking, lifetimes, and strict type rules, which are essential for ensuring memory safety and program correctness. These mechanisms form the foundation of `rustc`'s core checking logic and, as shown in our empirical study in Section 2, are frequent sources of suggestion bugs. To systematically test these critical components, SugBreaker designs mutation rules that deliberately target borrow, lifetime, and type constructs. This approach ensures that the generated programs exercise `rustc`'s key analysis modules, such as type and borrow checking, enabling effective detection and validation of fix suggestion bugs. Different from random program mutations that often fail early during parsing, our tailored rules reach deeper into `rustc`'s analysis pipeline, producing more detailed and useful diagnostic information. This focused approach allows SugBreaker to effectively detect and validate bugs in `rustc`'s fix suggestions, providing thorough evaluation of Rust-specific mechanisms.

3.2.2 Borrow-Related Rust Program Mutation. Borrowing is one of the central mechanisms in Rust's ownership system, designed to ensure memory safety without relying on garbage collection. Instead of transferring ownership, a program can temporarily borrow access to a value through references, which may be either immutable (&T) or mutable (&mut T). At any point in a program, `rustc` ensures that either one mutable reference or any number of immutable references can exist, and all references must remain valid. These constraints make borrow checking one of the most distinctive and challenging aspects of `rustc`, as it requires precise reasoning about program state and reference lifetimes. To violate the constraints of borrowing mechanisms, we define three specific mutation rules, and provide their summarized descriptions along with examples in Table 3.

Table 3. Description of borrow-related mutation rules designed in SugBreaker, along with code examples.

Mutation Rule	Description	Code Before Mutation	Code After Mutation
Duplicate Borrow Insertion	Inserting a duplicate mutable reference to the same variable.	let borrow = &mut value; println!("{}", borrow);	let borrow = &mut value; <u>let borrow_dup = &mut value;</u> println!("{}", borrow);
Closure Borrow Capture Violation	Removing the move keyword from a closure.	fn foo() ->Box<dyn Fn(u32) ->u32>{ let x = 0u32; Box::new(<u>move</u> y x + y) }	fn foo() ->Box<dyn Fn(u32) ->u32>{ let x = 0u32; Box::new(y x + y) }
Closure Mutable Borrow Violation	Replacing a mutable reference parameter in the closure with implicit capture.	fn bar(x: &mut i32) {} fn foo(a: &mut i32) { let mut value = <u> s: &mut i32 </u> { bar(s); }	fn bar(x: &mut i32) {} fn foo(a: &mut i32) { let mut value = <u> { bar(s);</u> }

Duplicate Borrow Insertion. According to Rust's ownership rules, at any program point t , a variable v may have either a single mutable reference or any number of immutable references [35].

Let $\mathcal{B}(v, t)$ denote the set of active borrows of v at time t ; the invariant enforced by `rustc` requires that $|\mathcal{B}_{\text{mut}}(v, t)| \leq 1$ and $|\mathcal{B}_{\text{imm}}(v, t)| \geq 0$. To violate this constraint, we design a mutation that introduces an additional mutable borrow immediately after an existing one. When such duplicated borrows are subsequently used in the program, `rustc` reports an error indicating that a variable has been borrowed mutably more than once, thereby triggering diagnostics from the borrow checker.

Closure Borrow Capture Violation. Closures in Rust can capture environment variables either by reference or by value. Let C denote a closure and $x \in \text{Env}(C)$ a variable in its lexical environment. Applying the `move` keyword transfers ownership of x into C , enforcing that subsequent accesses respect lifetime constraints. To violate this constraint, we define a mutation rule that removes the `move` keyword, causing x to be captured by reference instead of by value. By applying the transformation, the capture mode of x changes from $x \mapsto \text{Owned}$ to $x \mapsto \text{Ref}$. Subsequent accesses to x through the closure can violate Rust’s borrow constraints, causing `rustc` to emit a borrow-checking diagnostic and a potential fix suggestion.

Closure Mutable Borrow Violation. Closures may accept mutable references as parameters to modify variables. Let a closure C take a mutable parameter $p : \&\text{mut } T$; `rustc` enforces the invariant that, at any program point t , there is at most one active mutable borrow of p . To violate this constraint, we define a mutation that removes the mutable parameter from the closure, effectively preventing C from taking p as a mutable reference. Formally, the type of C transforms from $C : (\&\text{mut } T) \rightarrow \tau$, where τ denotes the return type of the closure, to $C : () \rightarrow \tau$. Subsequent attempts to mutate the originally borrowed variable through C break the mutable borrow invariant, leading `rustc` to produce a borrow-checking error and may trigger the corresponding suggestion.

3.2.3 Lifetime-Related Rust Program Mutation. In Rust, lifetimes annotate the scope during which a reference is valid. Each reference has an associated lifetime, either explicitly declared (e.g., `'a`) or inferred by `rustc`. Lifetimes ensure memory safety by preventing dangling references and use-after-free errors. Rust enforces that references cannot outlive the data they point to, and that function signatures respect input-output lifetime relationships. Violations occur when these constraints are not satisfied, such as returning a reference to a local variable or assigning a reference with a shorter lifetime to a location expecting a longer lifetime. To deliberately violate the lifetime constraints, we define three mutation rules, summarized with examples in Table 4.

Table 4. Description of lifetime-related mutation rules designed in SugBreaker, along with code examples.

Mutation Rule	Description	Code Before Mutation	Code After Mutation
Lifetime Annotation Injection	Injecting duplicate lifetimes in <code>impl</code> blocks for structs or traits.	<pre>struct myStruct<'a> { a:&'a i32 } impl<'a> myStruct<'a> { fn method(&self, x:&'a i32) {} }</pre>	<pre>struct myStruct<'a> { a:&'a i32 } impl<'a> myStruct<'a> { fn method<'a>(&self, x:&'a i32) {} }</pre>
Lifetime Constraint Removal	Removing an existing lifetime constraint from an <code>impl</code> block.	<pre>impl<'a: 'b, 'b, T> Trait<'a, 'b> for T { fn cast(&'a self) -> &'b T { self } }</pre>	<pre>impl<'a, 'b, T> Trait<'a, 'b> for T { fn cast(&'a self) -> &'b T { self } }</pre>
Lifetime Annotation Removal	Removing the lifetime annotation from a type, e.g., reference, struct, and enum.	<pre>struct Foo<'a>{ x: &'a bool }</pre>	<pre>struct Foo<'a>{ x: &bool }</pre>

Lifetime Annotation Injection. Let $\mathcal{L}(r)$ denote the declared lifetime of a reference r . Lifetime parameters specify the valid scope of references and ensure memory safety. To violate these constraints, we introduce a mutation that adds an extra, potentially redundant lifetime annotation

to r . This transformation can create inconsistencies or ambiguities in the set of lifetimes, formally $\mathcal{L}(r) \mapsto \mathcal{L}'(r)$, where $\mathcal{L}'$ may conflict with existing lifetimes. Subsequent use of r can then violate Rust's lifetime rules, prompting `rustc` to emit a diagnostic. Practically, this corresponds to adding an extra lifetime parameter to a function or method that already has a lifetime annotation.

Lifetime Constraint Removal. Let $\mathcal{R} = \{('a : 'b), ('b : 'c), \dots\}$ denote the set of lifetime constraints specifying that one lifetime outlives another. To violate these constraints, we define a mutation that removes a constraint from $\mathcal{R}$, e.g., $('a : 'b) \in \mathcal{R} \mapsto \mathcal{R}' = \mathcal{R} \setminus \{('a : 'b)\}$. By doing so, `rustc` can no longer verify that the relationships between lifetimes are preserved, and subsequent references may break the outlives invariant, triggering lifetime-checking errors. In practice, this mutation removes an explicit lifetime bound between two lifetimes in an `impl` block.

Lifetime Annotation Removal. For a reference r with declared lifetime $\mathcal{L}(r)$, removing the lifetime annotation yields an elided lifetime $\mathcal{L}'(r)$ inferred by `rustc`. This mutation changes $\mathcal{L}(r) \mapsto \mathcal{L}'(r)$ and may cause inconsistencies with other interacting references. As a result, `rustc` can detect violations of lifetime rules, producing errors that indicate missing or ambiguous lifetime information. In practice, this involves deleting lifetime annotations from a `struct` field, function parameter, or return type, leaving `rustc` to infer them automatically. Note that this rule does not always trigger an error, as `rustc` may correctly infer the lifetime when the annotation is redundant.

3.2.4 Type-Related Rust Program Mutation. Rust is a statically typed language, so the type of every variable and expression is known at compile time, enabling early detection of type errors. In addition to primitive and concrete types, Rust supports associated types, which act as placeholders within traits to define abstract relationships between types, and trait objects, which allow dynamic dispatch while ensuring the object implements a given trait. Correct use of associated types and trait object bounds is critical for type safety, and violations can result in compiler errors. To test these mechanisms, we define six specific mutation rules, summarized with examples in Table 5.

Illegal Drop Implementation Injection. In Rust, the `Drop` trait provides a destructor for types, and it can only be implemented for concrete types. Let $\mathcal{D}(T)$ denote the set of `Drop` implementations for type T . Rust enforces the invariant that a `Drop` implementation exists only for a valid concrete type and cannot be applied to traits or references. To violate this constraint, we define a mutation that injects a `Drop` implementation for an invalid target, e.g., a trait or a reference type. Formally, this changes $\mathcal{D}(T) \mapsto \mathcal{D}'(T) = \mathcal{D}(T) \cup \text{invalid drop}$, which can trigger `rustc` to produce a type-checking error. Practically, this mutation corresponds to adding a `Drop` implementation for a trait or a mutable reference to a type, which is not allowed by Rust's type system.

Duplicate Associated Type Injection. In Rust, a trait may declare associated types to specify abstract type relationships, and a type implementing a trait must provide at most one definition for each associated type. Let $\mathcal{A}(\tau)$ denote the set of associated types defined within trait τ . Rust enforces the invariant that each associated type identifier occurs at most once in $\mathcal{A}(\tau)$. To violate this constraint, we define a mutation that injects an additional associated type with the same identifier or conflicting type constraints, formally changing $\mathcal{A}(\tau) \mapsto \mathcal{A}'(\tau) = \mathcal{A}(\tau) \cup \text{duplicate type}$. This can lead `rustc` to emit type-checking errors due to conflicting or duplicate associated type definitions. Practically, this mutation corresponds to adding a duplicate associated type declaration in a trait, either with the same identifier or incompatible type bounds, violating associated type rules.

Trait Implementation Qualification Removal. In Rust, when accessing an associated type of a generic type implementing a trait, the type must be fully qualified to indicate the trait context, ensuring type correctness. Let $Q(T, \tau, \alpha)$ denote the qualified access of associated type α for type T under trait τ . Rust enforces the invariant that α is referenced as $Q(T, \tau, \alpha)$ to maintain clarity in type resolution. To violate this invariant, we define a mutation that removes the trait qualification, formally changing $Q(T, \tau, \alpha) \mapsto Q'(T, \alpha)$, which can cause `rustc` to produce a type-checking error.

Table 5. Description of type-related mutation rules designed in SugBreaker, along with code examples.

Mutation Rule	Description	Code Before Mutation	Code After Mutation
Illegal Drop Implementation Injection	Injecting a Drop implementation for a trait object or a mutable reference to a struct.	<pre>trait MyTrait {} struct MyStruct {}</pre>	<pre>trait MyTrait {} impl Drop for MyTrait { fn drop(&mut self) {} } struct MyStruct {} impl Drop for &'_ mut MyStruct { fn drop(&mut self) {} }</pre>
Duplicate Associated Type Injection	Injecting a duplicate associated type into a trait.	<pre>trait T1 {} trait Foo { type A: T1; }</pre>	<pre>trait T1 {} trait T2 {} trait Foo { type A: T1; } trait Bar: Foo { type A: T2; }</pre>
Trait Implementation Qualification Removal	Removing trait qualification from an associated type access.	<pre>pub trait Foo<T> { type A; fn get(&self, t: T) -> Self::A; } fn foo<I: for<'x> Foo<&'x isize>>> (x: <I as Foo<&isize>>::A) {}</pre>	<pre>pub trait Foo<T> { type A; fn get(&self, t: T) -> Self::A; } fn foo<I: for<'x> Foo<&'x isize>>> (x: <I>::A) {}</pre>
Reference Mutability Flipping	Flipping reference mutability in function parameters or variable declarations.	<pre>fn foo(a: &mut i32, b: &i32) {} fn main() { foo(&mut 1, &2); }</pre>	<pre>fn foo(a: &i32, b: &mut i32) {} fn main() { foo(&1, &mut 2); }</pre>
Type Annotation Replacement	Replacing the type annotation of a variable or function parameter.	<pre>fn plus_one(x: i32) -> i32 { x + 1 } fn main() { let y = plus_one(5); }</pre>	<pre>fn plus_one(x: bool) -> i32 { x + 1 } fn main() { let y = plus_one(5); }</pre>
Trait Impl Method Violation	Adding a method to a trait implementation that is not declared in the trait.	<pre>trait myTrait { fn a(); } struct myStruct; impl myTrait for myStruct { fn a() {} }</pre>	<pre>trait myTrait { fn a(); } struct myStruct; impl myTrait for myStruct { fn a() {} fn b() {} }</pre>

due to ambiguity in resolving the associated type. Practically, this mutation corresponds to deleting the trait part of a qualified associated type access, e.g., as shown in Table 5, transforming `<I as Foo<&isize>::A` to `<I>::A`, thus introducing ambiguity in type resolution.

Reference Mutability Flipping. In Rust, references are either mutable or immutable, with `rustc` enforcing that mutable references are unique while immutable ones can coexist. Let $\mathcal{R}(v)$ denote the mutability of a reference to variable v , where $\mathcal{R}(v) = \text{mut}$ indicates mutable and $\mathcal{R}(v) = \text{imm}$ indicates immutable. Rust requires that each reference is accessed according to its declared mutability. To violate this constraint, we define a mutation that flips reference mutability, formally $\mathcal{R}(v) \mapsto \mathcal{R}'(v)$, where $\mathcal{R}'(v)$ is the opposite of the original. This can trigger borrow-checking errors when the program uses references inconsistently. Practically, the mutation swaps mutable and immutable references in function parameters and corresponding call arguments, deliberately breaking Rust's reference constraints.

Type Annotation Replacement. In Rust, function parameters and variables are statically typed, and the compiler enforces that the types match their usage to ensure type safety. Let $\mathcal{T}(v)$ denote the declared type of variable or parameter v . Rust enforces the invariant that the value assigned to or passed as v conforms to $\mathcal{T}(v)$. To violate this invariant, we define a mutation that replaces the original type annotation with a different, incompatible type, formally changing $\mathcal{T}(v) \mapsto \mathcal{T}'(v)$,

where $\mathcal{T}'(v) \neq \mathcal{T}(v)$. This can cause `rustc` to produce a type-checking error when the program attempts to use the variable inconsistently with its new type. Practically, this mutation corresponds to transforming a function parameter or variable type to a different type. For example, as shown in Table 5, the `i32` type is replaced with `bool`, introducing a type mismatch error.

Trait Impl Method Violation. In Rust, trait implementations must define only the methods declared in the trait. The type system enforces that all required methods are implemented and prohibits undeclared methods. Let $\mathcal{M}(T, \tau)$ denote the set of methods implemented by a type T for trait τ , and let $\mathcal{M}_\tau$ be the set of methods declared in τ . Rust enforces the invariant $\mathcal{M}(T, \tau) \subseteq \mathcal{M}_\tau$. To violate this rule, we design a mutation that adds an extra method $m \notin \mathcal{M}_\tau$, producing $\mathcal{M}'(T, \tau) = \mathcal{M}(T, \tau) \cup m$. This triggers a type-checking error, indicating that the implementation includes methods not declared in the trait. Practically, this mutation adds a method to a type's trait implementation that is not specified in the trait, deliberately violating Rust's implementation rules.

3.3 Detecting Bugs in `rustc`'s Fix Suggestions

After generating mutated Rust programs, `SugBreaker` feeds them to `rustc` and analyzes its feedback. In this section, we describe how `SugBreaker` detects bugs from `rustc`'s responses and how these bugs are further classified.

3.3.1 Bug Detection. While Rust's official package manager, Cargo, provides the `cargo fix` command to apply suggestions automatically, it only handles those classified as `MachineApplicable`. Each suggestion from `rustc` has an applicability level (`MachineApplicable`, `MaybeIncorrect`, `HasPlaceholders`, or `Unspecified`), so many suggestions are not applied by Cargo and cannot be tested. To address this, we develop custom scripts that parse all suggestions and apply them to the corresponding source locations for comprehensive validation. `SugBreaker` invokes `rustc` with the JSON output flag to capture suggestions in a structured format. For each suggestion, it extracts precise location information, including file path, line, and column. It determines whether the fix requires adding, deleting, or replacing code and constructs a mapping from each suggestion to its corresponding source code. Based on the extracted information, `SugBreaker` applies the fix and recompiles the program. If compilation succeeds, the suggestion is considered correct and the program is added to the seed corpus for the next mutation iteration. Otherwise, the case may indicate a potential `rustc` bug that warrants further investigation.

3.3.2 Bug Classification. To detect `rustc` bugs, particularly suggestion bugs, we establish criteria based on the behavior of programs after applying mutations and the suggested fixes.

(1) `rustc`'s Fix Suggestion Bugs. To systematically detect `rustc`'s fix suggestion bugs, we adopt a multi-round validation process. For each mutated program that triggers a `rustc` fix suggestion, we iteratively apply the suggested fix and recompile, repeating this process for up to m rounds or until no further suggestions are produced. For `SugBreaker`, we set $m = 6$, since we observe that if `rustc` still cannot produce a successful fix after six iterations, the original error is generally unresolvable, either because the suggestion is misdirected or the applied fix introduces new errors without further guidance. After this process, suggestion bugs are classified into the following categories:

- **Cyclic Suggestions.** A suggestion is considered cyclic if applying it across iterations leads to contradictions. For example, `rustc` may first suggest deleting a variable and later suggest adding it back. Such cycles indicate at least one incorrect suggestion, possibly caused by incomplete variable analysis or interactions between compiler passes, etc.
- **Unidentifiable Fix Location.** Although `rustc` provides file path, code line, and column for each suggestion, some fixes cannot be applied successfully. Because the location may not

match `rustc`'s intended modification or falls outside user-level source code, such as internal `rustc` libraries or macro expansions. This symptom is recorded as a `rustc` fix suggestion bug.

- **Ineffective Suggestions.** A suggestion is considered ineffective if, after the multi-round application process, the program still fails to compile with the same error. Such cases indicate that the suggestion failed to address the underlying issue, reflecting inaccuracies in `rustc`'s analysis or its inability to generate a correct fix.
- **Suspicious Suggestions.** A suggestion is considered suspicious if applying it leads to a different compilation error. In these cases, it is unclear whether the suggestion partially fixed the original error, introduced a new error, or is incorrect. Automated validation cannot reliably determine correctness, so suspicious suggestions are flagged for manual inspection.

(2) Performance Issues. Our hypothesis for `rustc` testing is that injecting type-related and ownership-related violations into valid Rust programs should not cause abnormally long compilation times. Since our constraint-violation-guided mutations are unlikely to introduce infinite loops, any compilation timeout is treated as a potential performance bug. To reduce false positives, we set a timeout threshold of 10 seconds, flagging only cases that exceed this limit.

(3) Crash. In addition to bugs in the fix suggestions, compiler crashes are another type of bug revealed during our testing. We categorize crashes into two types: internal compiler errors (ICEs), where `rustc` panics during compilation, and external crashes, such as segmentation faults triggered by the backend LLVM. Since our mutations do not inject macros like `panic!()` or `todo!()`, any crash triggered by a mutated program is unexpected and recorded as a crash bug.

4 Experiment Design

In this section, we introduce the experiment setup, including research questions and baseline tools. To conduct the experiments, we implement `SugBreaker` upon Rust [37]. All experiments were conducted on a server running Ubuntu 18.04.3 LTS. The hardware was equipped with an Intel Xeon Platinum 8352V CPU (16 Cores @ 2.10 GHz) and 32 GB of RAM.

4.1 Research Questions

`SugBreaker` is designed to mutate Rust programs and systematically test the fix suggestions generated by `rustc`. To this end, we empirically explore the following research questions (RQ):

RQ1 (Effectiveness): How effective is `SugBreaker` in testing the fix suggestions generated by `rustc`? In this RQ, we evaluate whether `SugBreaker` can effectively reveal errors in `rustc`'s fix suggestions. We apply `SugBreaker` to the latest `rustc` version and report the number of bugs discovered, indicating which have been confirmed or fixed by developers. We also assess the code coverage of `SugBreaker`-generated test programs on `rustc` and compare it with baseline tools to evaluate testing adequacy.

RQ2 (Mutation Rules): How effective are different mutation rules in testing various components of `rustc`? To evaluate the effectiveness of our designed mutation rules, we separately examine the impact of three categories of rules on generating test cases. We then compare the code coverage of these generated test cases with that of the original seed programs, with a particular focus on the coverage improvements within several core modules of `rustc`.

RQ3 (Efficiency): How efficient is `SugBreaker` compared to baseline approaches? In this RQ, we aim to assess the efficiency of `SugBreaker` in comparison with other baseline tools. We first evaluate the code coverage of `rustc` achieved within a fixed testing time, and more importantly, the suggestion trigger rate of `rustc`. We then report the efficiency of test case generation for each tool, as well as the time cost of testing `rustc`.

RQ4 (Case Study): Analysis of the bug types detected by SugBreaker. As the first study focusing on testing fix suggestions generated by rustc, we investigate the triggering behaviors of rustc’s suggestion bugs and analyze their underlying causes. This analysis provides insights into some of the challenges faced by the Rust compiler. We hope that these findings can guide future work on automated program repair and contribute to the development of more reliable compiler-assisted fixes in Rust.

4.2 Baseline Approaches

To the best of our knowledge, there are few approaches proposed for testing the diagnostics system of compilers. Epiphron [31] and DIPROM [34] are proposed to detect warning defects of GCC and Clang. Since these approaches are designed for random program generation and mutation based on the syntax rules of C/C++, they cannot be directly applied to Rust programs. Moreover, their assumptions regarding compiler directives and conditions specific to C/C++ compilers do not hold for the Rust compiler. Therefore, we did not select them as baselines. Considering the unique characteristics of Rust’s syntax and type system, we instead focus on comparing SugBreaker with existing tools specifically designed for testing rustc. In particular, we briefly describe the baseline techniques used for comparison in our study as follows.

(1) *Fuzz-rustc* [24]. Fuzz-rustc adapts LibFuzzer [19] to mutate the input byte stream and generate tests. It primarily aims to discover crash bugs by fuzzing various program inputs.

(2) *Tree-splicer* [1]. Tree-splicer generates new test programs by extracting AST fragments from existing seed programs and recombining them into novel programs. Its testing focus is on increasing structural diversity to explore potential compiler errors, such as crashes.

(3) *ICEMaker* [12]. ICEMaker combines the techniques of Fuzz-rustc and Tree-splicer, performing iterative mutations on generated programs while leveraging analysis tools such as Miri [22] and Clippy [26]. ICEMaker is not only able to detect crashes but also uncovers miscompilation errors, providing a broader assessment of compiler correctness.

(4) *RustSmith* [28]. RustSmith constructs syntactically valid Rust programs by generating ASTs that strictly follow the Rust grammar. It focuses on detecting miscompilation or misoptimization by identifying inconsistencies across different rustc versions and optimization levels through differential testing, thus targeting rustc’s correctness and soundness.

(5) *Rustlantis* [40]. Rustlantis generates customized MIRs using the `mir!()` macro, directly manipulating rustc’s intermediate representation. Its testing focus is on exposing subtle bugs in MIR-level optimizations that may not be reachable through high-level Rust code.

There are also two tools designed for testing rustc that we did not include as baselines. First, *Rust-twins* [43] generates Rust programs relying on macros implemented within the rustc core. As a result, it is tied to the internal implementation of the specific rustc version under test, making it infeasible to interact with the latest rustc release or with an instrumented version for coverage collection, due to poor inter-version compatibility and potential changes in internal APIs that may prevent successful compilation. Second, *Typecheck-fuzzer* [6] is an early tool targeting Rust 1.0. Its source code is unavailable, and it was designed for the syntax and semantics of Rust 1.0, which differ significantly from modern Rust, making it unsuitable for comparison.

Experimental Setup. SugBreaker relies on seed programs for mutation, which we collected from Rust’s official test suite. We compiled all test cases with the latest rustc at the time (v1.87.0) and retained only those that compiled successfully, resulting in a seed set of 6,355 programs. The same set was used for all baseline tools that require seeds, including Fuzz-rustc, Tree-splice, and ICEMaker, to ensure fair comparison. For RQ1, RQ2, and RQ3, we also employed rustc v1.87.0, with coverage in RQ1 and RQ2 measured using an instrumented build of the same compiler.

5 Result Analysis

This section presents the experiment result and then analyzes the performance of our approach.

5.1 Answer to RQ1: Effectiveness

To evaluate the effectiveness of our approach, we applied SugBreaker to detect rustc's fix suggestion bugs in the latest version of rustc and submitted the identified issues. In addition, we recorded the line coverage of rustc during testing and compared it with that achieved by baseline approaches.

Table 6 summarizes the issues we submitted. Five have been fixed, six confirmed, and one submitted but not yet confirmed by the developers. Nine of the twelve issues are bugs in rustc fix suggestion. In addition, we also detected two ICE bugs and one performance issue where rustc failed to terminate, officially classified as a *Hang* by the rustc team. These results provide preliminary evidence that SugBreaker can identify real bugs in rustc's fix suggestions. We also record the error index for each triggered bug. The rustc team maintains over 500 error indices, each linked to an illustrative example [36], although many shallow syntax errors lack an index. Our generated test cases triggered a variety of error indices, indicating diverse underlying causes and further confirming SugBreaker's effectiveness. For the fixed issues, we examined the corresponding pull requests to identify affected modules. Among the five fixed cases, one involved HIR analysis, one the MIR borrow checker, and three HIR-based type checking. This suggests that SugBreaker can trigger bugs in Rust-specific core modules, which is valuable for improving rustc's reliability.

Table 7 reports the coverage achieved by SugBreaker and baseline tools on rustc's core modules. Following the classification in Section 2, we divide the core components into parser, type checking, borrow checking, and diagnostics, with the corresponding directories listed in Table 7. SugBreaker achieves the highest coverage in most modules, demonstrating that its mutation strategies effectively exercise rustc's core checking modules and facilitate the detection of suggestion bugs. Overall, SugBreaker's coverage is about 1% lower than Tree-splicer, which randomly splices AST nodes and thus attains higher coverage in the AST module. However, the AST module corresponds to early compilation stages, whereas our focus is on HIR-based and MIR-based analysis, where suggestion bugs are more likely to occur. Coverage for Rustlantis and RustSmith is relatively low. Both are rule-based generators producing primarily valid programs, which exercise only a limited portion of rustc and result in lower line coverage.

5.2 Answer to RQ2: Mutation Rules

To evaluate the effectiveness of our mutation rules, we applied three categories to the same 6,355 seed programs used in RQ1, generating 5,000 test cases per category. We executed these test cases on rustc to measure code coverage and assess the impact of each mutation type. By comparing the coverage achieved by the different mutations, we can quantify how each rule category contributes to exploring rustc's behaviors and uncovering potential suggestion bugs.

Table 6. Status of the reported rustc bugs.

	Status	Bug Type	Error Index
1	Fixed	ICE	-
2	Fixed	Suggestion Error	E0597
3	Fixed	Suggestion Error	E0412
4	Fixed	Suggestion Error	E0308
5	Fixed	Suggestion Error	E0308
6	Confirmed	Suggestion Error	E0599
7	Confirmed	Hang	E0207
8	Confirmed	ICE	-
9	Confirmed	Suggestion Error	-
10	Confirmed	Suggestion Error	E0107
11	Confirmed	Suggestion Error	E0412
12	Reported	Suggestion Error	E0091, E0207

Table 7. Comparison of rustc's line coverage by SugBreaker and baseline tools across key compiler components (AST, HIR, MIR, and error reporting).

Module		Fuzz-rustc	Tree-splicer	ICEMaker	Rustlantis	RustSmith	SugBreaker
Parser	rustc_ast	71.11%	72.66%	45.24%	30.52%	27.74%	64.83%
	rustc_hir	62.81%	73.72%	47.98%	20.24%	20.06%	68.92%
Type Checking	rustc_infer	84.10%	87.29%	64.84%	54.22%	56.63%	84.98%
	rustc_trait	52.17%	66.90%	43.80%	16.65%	16.16%	67.68%
	rustc_type	74.13%	73.63%	53.72%	43.67%	43.67%	75.08%
Borrow Checking	rustc_middle	61.09%	70.78%	63.37%	39.86%	39.89%	71.82%
	rustc_mir	59.27%	53.81%	48.24%	41.31%	33.97%	57.12%
	rustc_borrowck	33.92%	50.52%	28.22%	20.01%	26.32%	61.18%
Diagnostics	rustc_error	72.93%	74.67%	75.11%	67.25%	43.23%	75.11%
	rustc_errors	59.73%	62.72%	58.88%	39.93%	40.18%	66.99%
Total		55.36%	58.24%	45.59%	29.61%	28.61%	57.51%

Table 8 shows coverage results for core rustc modules and diagnostic submodules under different mutation rules, along with improvements over the original seed set. All three mutation categories increase rustc coverage, demonstrating the effectiveness of each rule set. Lifetime-related rules mainly improve coverage in the AST and error reporting modules, as lifetime annotations are analyzed during parsing and violations are typically detected immediately. Borrow-related rules enhance coverage in the MIR builder, borrow checking, trait solving, and borrow diagnostics, reaching more code associated with borrow and trait handling. Type-related rules increase coverage across type checking, borrow checking, and error reporting due to the larger number of type mutations and their presence at multiple stages, resulting in higher coverage and suggestion-triggering rates. Lifetime rules primarily affect the AST module, reflecting parser-level scenarios triggered by the position of lifetime annotations, such as 'static. In contrast, borrow-related and type-related rules target HIR- and MIR-based checking modules and error reporting. By combining lifetime mutations with type or borrow transformations over multiple rounds, SugBreaker further increases coverage, testing diversity, and suggestion-triggering in core rustc modules.

Table 8. Coverage of core rustc modules and diagnostic submodules with different mutation rules enabled, along with improvements over the seed set.

Module	Seed	Lifetime		Borrow		Type	
	cov	cov	Δ cov	cov	Δ cov	cov	Δ cov
rustc_ast_passes	59.59%	61.40%	+1.81%	59.59%	-	59.59%	-
rustc_hir_analysis	59.16%	59.34%	+0.18%	59.47%	+0.31%	68.24%	+9.08%
rustc_hir_typeck	52.75%	52.75%	-	58.18%	+5.43%	65.65%	+12.90%
rustc_infer/src/infer	81.85%	82.14%	+0.29%	83.94%	+2.09%	85.38%	+3.53%
rustc_middle/src/traits	51.58%	51.58%	-	58.30%	+6.72%	64.82%	+13.24%
rustc_middle/src/ty	69.91%	70.04%	+0.13%	72.59%	+2.68%	76.31%	+6.40%
rustc_mir_build/src/builder	80.88%	80.88%	-	84.12%	+3.24%	84.16%	+3.28%
rustc_borrowck	65.82%	65.96%	+0.14%	71.69%	+5.87%	71.31%	+5.49%
rustc_errors	58.88%	60.72%	+1.84%	65.10%	+6.22%	65.90%	+7.02%
rustc_hir_analysis/src/errors	0%	0%	-	0%	-	44.62%	+44.62%
rustc_trait_selection/src/error_reporting	0%	0%	-	80.95%	+80.95%	80.95%	+80.95%
rustc_borrowck/src/diagnostics	1.00%	1.00%	-	29.58%	+28.58%	34.11%	+33.11%

5.3 Answer to RQ3: Efficiency

To evaluate efficiency, we record the increase in `rustc` line coverage over time for test cases generated by each tool, as shown in Figure 3(a). `Rustlantis` generates MIR wrapped in macros, exercising only a few modules, mostly MIR-based optimization passes. `RustSmith` produces programs limited to primitive types and basic control structures, without higher-level Rust features such as generics. In contrast, the other tools achieve significantly higher coverage. `Tree-splicer` and `SugBreaker` perform similarly by extending existing test cases to generate diverse programs. `Fuzz-rustc` improves coverage by randomly mutating binaries. Although `Tree-splicer` achieves slightly higher overall coverage, as shown in Table 7, its random AST combinations often trigger parser-stage errors, preventing execution from reaching the core checking modules.

Figure 3(b) shows the trend of suggestion triggers as the number of generated test cases increases. To ensure fairness, we use the same `rustc` version and default compilation command without additional flags. The results demonstrate that `SugBreaker` consistently achieves the highest suggestion trigger rate, indicating its strong potential to expose suggestion bugs. In contrast, although `Fuzz-rustc` achieves relatively high coverage, as shown in Figure 3(a), its suggestion trigger rate is not as high as `SugBreaker` and `Tree-splicer`. This is likely because its randomly mutated binaries often produce invalid programs with many syntax errors, preventing `rustc` from generating suggestions. Moreover, our inspection reveals that `Rustlantis` produces only valid programs, and the suggestions it triggers are mostly warnings, such as those for unused variables or dead code. Similarly, `RustSmith` also generates mostly valid programs without warnings or suggestions, with only a few invalid programs triggering error reports and related suggestions.

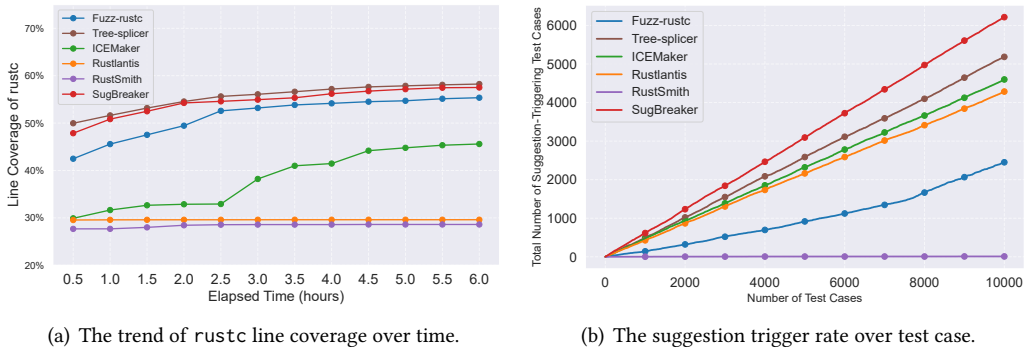


Fig. 3. Efficiency comparison between `SugBreaker` and the baseline approaches.

Table 9 reports the time required to generate and validate 1,000 test cases. Since `Tree-splicer` and `RustSmith` are program generators without automated validation, we classify the tools as either *testing tools* or *program generators*. `Fuzz-rustc` is excluded because it mutates binaries and invokes `rustc` through callbacks, bypassing part of the standard workflow and making timing results incomparable. For testing tools, most execution time is spent on validation rather than generation, as compiling with `rustc` is inherently costly. `SugBreaker` achieves the fastest generation speed and more efficient validation than `Rustlantis`. `ICEMaker` validates faster by compiling each test case only once, but can detect only `rustc` crashes. `Rustlantis` requires multiple compilations under different optimization levels to reveal MIR-level errors, while `SugBreaker` repeatedly applies and recompiles suggestions to detect incorrect fixes. These extra checks increase the cost but allow detection of more complex bug types. Among program generators, `Tree-splicer` is generally fast,

Table 9. Time cost for program generation and rustc testing across baseline methods and SugBreaker.

rustc Testing Tools				Rust Program Generators	
Tool	Generation (s)	Testing (s)	Total (s)	Tool	Generation (s)
Rustlantis	529.3	10396.02	10925.32	Tree-splicer	388.27
ICEMaker	12.84	123.2	136.04	RustSmith	600.5
SugBreaker	9.13	89.58	98.71	Fuzz-rustc	-

but its random AST composition can occasionally cause timeouts, as confirmed by ICEMaker [13]. We set a timeout threshold of 60s, after which the average generation time per test case increases significantly. Rustsmith is slower because each generated program typically exceeds 2,000 lines, and the process manages hundreds of variables, increasing the time needed per test case.

5.4 Answer to RQ4: Case Study

In this section, we examine the bugs detected by SugBreaker and analyze their potential root causes. Figure 4 presents several real-world code bugs detected by SugBreaker.

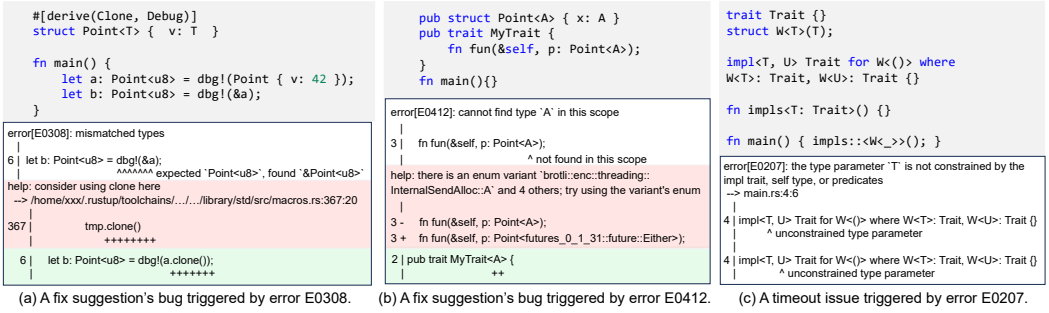


Fig. 4. Code examples detected by SugBreaker, each revealed by programs that trigger different errors.

Bug Case 1: The test case in Figure 4 (a) defines a generic `Point` struct. In `main`, variable `a` of type `Point<u8>` is created, and variable `b` is initialized with `dbg!(&a)`. Then, `rustc` reports a type mismatch because `dbg!(&a)` produces `&Point<u8>`, while `b` expects an owned `Point<u8>`, violating Rust's ownership rules. Although the error message is correct, `rustc`'s suggested fix is misleading, pointing to the standard library, which users cannot and should not modify. The appropriate fix is cloning `a` in `dbg!`, i.e., `dbg!(a.clone())`, producing a value that matches variable `b`'s type and allowing the program to compile. Such misleading suggestions may arise from `rustc`'s error localization. During compilation, `rustc` first expands macros and then analyzes the resulting code. As a result, error localization can point to the internal implementation of a macro rather than the relevant user code, causing the suggestion engine to propose fixes in unrelated locations. This case highlights a key challenge for `rustc` in generating accurate suggestions, since identifying the actual source of an error in macro-expanded or generic code is difficult, and mislocalization can easily lead to misleading fix suggestions.

Bug Case 2: The test program in Figure 4 (b) defines a generic `Point` struct and a trait `MyTrait` with a method `fun` taking a `Point<A>` parameter. The generic type `A` is not declared in the trait, causing `rustc` to report that `A` is not in scope. In Rust, a generic type must be declared where it is used, such as in a trait, function, or `impl` block. However, `rustc`'s suggested fix is misleading,

recommending an unrelated enum variant from the standard library. Note that `rustc` actually reports four similar suggestions; due to space limitations, only one is shown in the figure. All of the suggested enum variants are unrelated to the context of this program. The correct fix should advise adding `A` to `MyTrait`'s definition. Interestingly, renaming `A` to another name allows `rustc` to produce the correct suggestion. This behavior likely arises because the suggestion engine tends to prioritize similarly named types in scope without fully considering the intended generic context. This indicates that the current heuristics for generating fix suggestions may rely more on name similarity than on a deeper analysis of type relationships and usage context. While this approach usually provides helpful guidance, it can sometimes result in irrelevant suggestions, particularly in cases involving undeclared generics or complex trait hierarchies.

Bug Case 3: Although `SugBreaker` is primarily designed to detect `rustc`'s fix suggestion bugs, it can also trigger other compiler errors, such as crashes or non-terminating compilation. Figure 4 (c) illustrates a non-terminating case. The test program contains several syntax and type errors. First, the type parameters `T` and `U` in the `impl` are unused in `W<()>`, which is not allowed in Rust. Second, the recursive where-clause creates an invalid self-referential constraint that cannot be satisfied. Third, using a type placeholder as a generic type is not allowed in Rust. However, instead of reporting errors, `rustc` enters a non-terminating compilation process. This bug has been confirmed by `rustc` developers, but the corresponding fix has not yet been established. It is likely related to a potential infinite loop during trait resolution, which may occur when `rustc` repeatedly attempts to resolve complex or recursive trait bounds without reaching a stable solution. Such situations can prevent `rustc` from producing accurate error messages and highlight subtle challenges in the trait solver's design. This case demonstrates that `SugBreaker`'s mutation method can generate high-quality invalid programs that effectively exercise core `rustc` modules, such as the trait solver.

6 Discussion

In this section, we first discuss the novel aspects of `SugBreaker` compared to existing `rustc` testing tools, then explore potential extensions of our constraint-violation-guided mutation approach, and finally discuss threats to validity.

6.1 Comparison with Existing `rustc` Testing Tools

In recent years, both the open-source community and academia have developed tools for testing `rustc`. Popular fuzzers such as `ICEMaker` [12], `Tree-splicer` [1], and `Fuzz-rustc` [24] mainly target crash bugs and cannot assess compilation correctness. Academic efforts include `RustSmith` [28], which generates syntactically valid Rust code and applies differential testing to reveal potential miscompilation bugs. `Rustlantis` [40] focuses on MIR generation and differential testing for misoptimization. `Rust-Twins` [43] combines LLMs with Rust macros to create semantically equivalent programs for miscompilation testing. `Typecheck-fuzzer` [6], designed for Rust 1.0, evaluates the completeness of the type checker. Compared with existing tools, `SugBreaker` addresses a critical yet previously overlooked problem by systematically detecting bugs in `rustc`'s fix suggestions. Because these suggestions are directly exposed to users, flaws in them can mislead programming practices and propagate errors. By ensuring their correctness, `SugBreaker` not only improves compiler reliability and the users experience but also strengthens the trustworthiness of Rust's tooling ecosystem, thereby contributing to safer and more reliable Rust software development.

Beyond its focus on detecting bugs in fix suggestions, `SugBreaker` employs mutation strategies designed for Rust-specific mechanisms. This design naturally enables deeper interaction with `rustc`'s semantic analysis pipeline, which makes them more effective in testing `rustc`'s core checking modules. In contrast, existing fuzzers often rely on random program composition that frequently fails in the parsing stage, preventing deeper analysis. `Rustlantis` is restricted to MIR-level

testing and therefore overlooks HIR-based modules such as type checking and lifetime analysis. RustSmith focuses on syntactic validity but does not generate advanced constructs like generics. By leveraging Rust's unique language features, SugBreaker enhances the suggestion trigger rate and naturally drives exploration into deeper compiler modules within the semantic analysis pipeline.

6.2 Extending Constraint-Violation-Guided Mutation

Our proposed constraint-violation-guided mutation method is primarily designed to inject errors that violate type or borrow checking constraints into valid Rust programs, thus testing whether the suggestions produced by `rustc` conform to expectations. This targeted, scenario-specific mutation approach can be naturally extended to other similar testing contexts. For instance, an empirical study [18] reported that approximately 20% of `rustc` bugs are related to Rust's unstable features. Other studies [14, 15] have reported that libraries may incorporate buggy unstable features, enabling those bugs to propagate along dependency chains across the dependency graph. In practice, unstable features often correspond to a syntactic structure that is applicable only in certain contexts, and the usage typically follows clear and consistent patterns. By designing mutation rules that intentionally violate the constraints dictated by the specifications of these features, the constraint-violation-guided mutation could be leveraged to uncover `rustc` bugs associated with unstable features.

In addition, the constraint-violation-guided mutation approach can also be applied to test sanitizers, which are crucial for detecting security vulnerabilities caused by undefined behavior (UB). Li et al. [17] showed with UBFuzz that sanitizers contain many overlooked bugs, detecting 31 new issues and revealing frequent under-reporting. Sanitizers insert runtime checks during compilation, and flaws in these checks can lead to false positives or false negatives. Similar to `rustc`'s type and borrow checkers, sanitizers must correctly handle diverse program scenarios. By extending our proposed constraint-violation mutation method, we can generate Rust programs with intentional UB. Such programs often involve unsafe code [16, 21, 45], introducing new constructs and constraints for testing the effectiveness of sanitizers. Similarly, Rust programs mutated to violate safety constraints can be leveraged to test Rust-specific static analysis tools, such as Miri [10, 22], an official Rust tool maintained by the Rust team for detecting undefined behavior.

6.3 Threats to Validity

The threats to validity mainly lie in two aspects. First, the effectiveness of SugBreaker in testing `rustc`'s suggestions relies on the quality of seed programs. If a seed is too simple and lacks Rust-specific constructs such as structs, traits, or references, some mutation rules cannot be applied, limiting the evaluation of `rustc`'s core checkers. To mitigate this threat, we employ all valid Rust programs from `rustc`'s test suite as seed program sets. These programs cover a wide range of Rust features, providing sufficient diversity for SugBreaker to exercise `rustc`'s suggestion mechanisms, even if each seed program is not very long. The Second threat is the potential for false positives during validation. After the initial compilation, `rustc` may produce an error and a corresponding fix suggestion. Applying this suggestion can still result in a compilation failure with a different error, making it difficult to determine the fix suggestion's correctness. To reduce this threat, SugBreaker performs multiple rounds of iterative validation. If the new error differs from the previous one, it is not classified as a bug in the fix suggestion. Only if errors and suggestions persist after six iterations does SugBreaker mark the case as a potential bug for manual inspection.

7 Related Work

In this section, we discuss related work from three aspects.

Testing compiler diagnostic systems. Compiler testing has long been a major research focus, with extensive efforts devoted to enhancing the reliability and robustness of compiler implementations. Most of this prior work has concentrated on identifying crashes, miscompilations, or other correctness issues in widely used compilers such as GCC and Clang. To the best of our knowledge, only a few studies have specifically examined compiler diagnostic systems, and these primarily target warning defects rather than error-fixing suggestions. Sun et al.[31] introduced Epiphron, a method that detects warning defects by randomly generating C programs covering a wide range of language constructs and applying differential testing to compare compiler warnings across different compilers, versions, and optimization strategies. More recently, Tang et al.[34] proposed DIPROM, a diversity-guided mutation approach that constructs warning-sensitive programs to improve detection effectiveness and uncover subtle inconsistencies in compiler diagnostics. While these studies focus on compiler warnings, our work examines the fix suggestions provided of `rustc`. By evaluating the correctness and reliability of these suggestions, we aim to explore a complementary and underexamined aspect of compiler quality, particularly in the context of Rust’s unique language features and strict safety rules.

Testing the Rust compiler. The official Rust documentation [25] highlights three open-source fuzzing tools for `rustc` that mainly target compiler crashes: Fuzz-rustc [24], ICEMaker [12], and Tree-splicer [1]. Fuzz-rustc adapts LibFuzzer [19] to mutate input byte streams, Tree-splicer combines AST fragments from seed programs, and ICEMaker integrates both approaches with iterative mutations, also applying tools such as Miri [22] and Clippy [26]. In academia, several tools generate Rust programs with different strategies. RustSmith [28] builds syntactically valid programs and uses differential testing across compiler versions and optimization levels. Rustlantis [40] generates customized MIRs with the `mir!()` macro to detect MIR-level optimization errors. Rust-twins [43] creates semantically equivalent program pairs with macros assisted by LLMs to check for inconsistencies in HIR and MIR. Typecheck-fuzzer [6] employs constraint logic programming to construct well-typed programs for testing the type checker. Unlike these program generation approaches that aim to satisfy Rust’s type and borrow rules, our method mutates valid programs to deliberately introduce violations. This design enables us to exercise Rust’s safety checkers and evaluate the correctness of `rustc`’s fix suggestions.

Rust program generation. A major challenge of our approach is generating Rust programs that can effectively trigger diagnostics. While program generation has been widely studied, Rust’s ownership, lifetime model, and strict type rules introduce unique difficulties for both mutation and generation. RusSOL [7] generates programs that satisfy both functional specifications and Rust’s type rules. RUG [4] leverages LLMs to produce unit tests that conform to ownership and typing. Yin et al. [44] and Jiang et al. [9] focus on generating valid API call sequences, while tools such as Crabtree [32], SyRust [33], RuMono [46], and RPG [42] extend this to support generics and more complex type structures. For compiler testing, Rust-twins [43] creates semantically equivalent programs using macros, and RustSmith [28] constructs ASTs by rules to produce valid code. In contrast to these techniques, which focus on generating programs that pass Rust’s checks, our method mutates valid programs to deliberately introduce violations. This allows us to exercise Rust’s safety checkers and evaluate `rustc`’s fix suggestions.

8 Conclusion

In this paper, we presented SugBreaker, an automated framework for evaluating `rustc`’s fix suggestions by injecting type-related, borrow-related, and lifetime-related mutations into valid Rust programs. On the latest `rustc` version, SugBreaker has already detected 11 confirmed or fixed bugs, including 9 suggestion bugs. Compared with the baseline `rustc` testing tools, SugBreaker covers

more of rustc's core checking modules and achieves a higher suggestion trigger rate, demonstrating its effectiveness and efficiency in systematically testing rustc's suggestion mechanisms.

Acknowledgments

We would like to thank the anonymous reviewers for their constructive comments. This work was supported by the National Natural Science Foundation of China under Grant No. 62372225.

References

- [1] Langston Barrett. 2023. langston-barrett/tree-splicer: Simple grammar-based test case generator. <https://github.com/langston-barrett/tree-splicer>. (Accessed on 12/05/2023).
- [2] William Bugden and Ayman Alahmar. 2022. Rust: The programming language for safety and performance. *arXiv preprint arXiv:2206.05503* (2022).
- [3] William Bugden and Ayman Alahmar. 2022. Rust: The Programming Language for Safety and Performance. *arXiv:2206.05503* [cs.PL] <https://arxiv.org/abs/2206.05503>
- [4] Xiang Cheng, Fan Sang, Yizhuo Zhai, Xiaokuan Zhang, and Taesoo Kim. 2025. Rug: Turbo Llm for Rust Unit Test Generation. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 2983–2995. <https://doi.org/10.1109/ICSE55347.2025.00097>
- [5] Cloudflare. 2023. Cloudflare - The Web Performance & Security Company. <https://www.cloudflare.com/>. (Accessed on 12/06/2023).
- [6] Kyle Dewey, Jared Roesch, and Ben Hardekopf. 2015. Fuzzing the Rust Typechecker Using CLP (T). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 482–493. <https://doi.org/10.1109/ASE.2015.65>
- [7] Jonás Fiala, Shachar Itzhaky, Peter Müller, Nadia Polikarpova, and Ilya Sergey. 2023. Leveraging Rust Types for Program Synthesis. *Proc. ACM Program. Lang.* 7, PLDI, Article 164 (June 2023), 24 pages. <https://doi.org/10.1145/3591278>
- [8] Kelsey R. Fulton, Anna Chan, Daniel Votipka, Michael Hicks, and Michelle L. Mazurek. 2021. Benefits and drawbacks of adopting a secure programming language: rust as a case study. In *Proceedings of the Seventeenth USENIX Conference on Usable Privacy and Security (SOUPS'21)*. USENIX Association, USA, Article 31, 20 pages.
- [9] Jianfeng Jiang, Hui Xu, and Yangfan Zhou. 2022. RULF: rust library fuzzing via API dependency graph traversal. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (Melbourne, Australia) (ASE '21)*. IEEE Press, 581–592. <https://doi.org/10.1109/ASE51524.2021.9678813>
- [10] Ralf Jung, Hoang-Hai Dang, Jeehoon Kang, and Derek Dreyer. 2019. Stacked borrows: an aliasing model for Rust. *Proc. ACM Program. Lang.* 4, POPL, Article 41 (Dec. 2019), 32 pages. <https://doi.org/10.1145/3371109>
- [11] Steve Klabnik and Carol Nichols. 2023. *The Rust programming language*. No Starch Press.
- [12] Matthias Krüger. 2020. matthiaskrgr/icemaker: automatically find crashes in the rust compiler & tooling. <https://github.com/matthiaskrgr/icemaker>. (Accessed on 12/05/2023).
- [13] Matthias Krüger. 2023. How does fuzzing work to find a bug in icemaker. <https://github.com/matthiaskrgr/icemaker/issues/47#issuecomment-1849053284> [Online; accessed 2025-09-12].
- [14] Chenghao Li, Yifei Wu, Wenbo Shen, Rui Chang, Chengwei Liu, and Yang Liu. 2025. Demystifying Rust Unstable Features at Ecosystem Scale: Evolution, Propagation, and Mitigation. *IEEE Trans. Softw. Eng.* 51, 4 (March 2025), 1284–1302. <https://doi.org/10.1109/TSE.2025.3550160>
- [15] Chenghao Li, Yifei Wu, Wenbo Shen, Zichen Zhao, Rui Chang, Chengwei Liu, Yang Liu, and Kui Ren. 2024. Demystifying Compiler Unstable Feature Usage and Impacts in the Rust Ecosystem. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal) (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 24, 13 pages. <https://doi.org/10.1145/3597503.3623352>
- [16] Shaowen Li and Hiroyuki Sato. 2025. SBD: Securing safe rust automatically from unsafe rust. *Sci. Comput. Program.* 243, C (April 2025), 18 pages. <https://doi.org/10.1016/j.scico.2025.103281>
- [17] Shaohua Li and Zhendong Su. 2024. UBFuzz: Finding Bugs in Sanitizer Implementations. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (La Jolla, CA, USA) (ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 435–449. <https://doi.org/10.1145/3617232.3624874>
- [18] Zixi Liu, Yang Feng, Yunbo Ni, Shaohua Li, Xizhe Yin, Qingkai Shi, Baowen Xu, and Zhendong Su. 2025. An Empirical Study of Bugs in the rustc Compiler. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 411 (April 2025), 28 pages. <https://doi.org/10.1145/3763800>
- [19] LLVM. 2023. libFuzzer – a library for coverage-guided fuzz testing. — LLVM 18.0.0git documentation. <https://llvm.org/docs/LibFuzzer.html>. (Accessed on 12/09/2023).

- [20] Nicholas D Matsakis and Felix S Klock. 2014. The rust language. In *Proceedings of the 2014 ACM SIGAda annual conference on High integrity language technology*. 103–104.
- [21] Yusuke Matsushita, Xavier Denis, Jacques-Henri Jourdan, and Derek Dreyer. 2022. RustHornBelt: a semantic foundation for functional verification of Rust programs with unsafe code. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (San Diego, CA, USA) (PLDI 2022). Association for Computing Machinery, New York, NY, USA, 841–856. <https://doi.org/10.1145/3519939.3523704>
- [22] Miri. 2023. rust-lang/miri: An interpreter for Rust’s mid-level intermediate representation. <https://github.com/rust-lang/miri>. (Accessed on 12/14/2023).
- [23] RedoxOS. 2023. Redox - Your Next(Gen) OS - Redox - Your Next(Gen) OS. <https://www.redox-os.org/>. (Accessed on 12/06/2023).
- [24] David Renshaw. 2019. dwrensha/fuzz-rustc: setup for fuzzing the Rust compiler. <https://github.com/dwrensha/fuzz-rustc>. (Accessed on 12/05/2023).
- [25] Rust. 2020. Fuzzing - Rust Compiler Development Guide. <https://rustc-dev-guide.rust-lang.org/fuzzing.html> [Online; accessed 2025-08-30].
- [26] Rust-clippy. 2023. rust-lang/rust-clippy: A bunch of lints to catch common mistakes and improve your Rust code. Book: <https://doc.rust-lang.org/clippy/>. <https://github.com/rust-lang/rust-clippy>. (Accessed on 12/14/2023).
- [27] Servo. 2023. Servo, the embeddable, independent, memory-safe, modular, parallel web rendering engine. <https://servo.org/>. (Accessed on 12/06/2023).
- [28] Mayank Sharma, Pingshi Yu, and Alastair F. Donaldson. 2023. RustSmith: Random Differential Compiler Testing for Rust. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2023)*. Association for Computing Machinery, New York, NY, USA, 1483–1486. <https://doi.org/10.1145/3597926.3604919>
- [29] Diane Beightol Stephens, Kyu Hyung Lee, and Mustakimur Rahman Khandaker. 2024. RustLIVE: Reducing the Learning Barriers of Rust Through Visualization. In *2024 IEEE Frontiers in Education Conference (FIE)*. 1–9. <https://doi.org/10.1109/FIE61694.2024.10893207>
- [30] STRATIS. 2023. Stratis Storage. <https://stratis-storage.github.io/>. (Accessed on 12/06/2023).
- [31] Chengnian Sun, Vu Le, and Zhendong Su. 2016. Finding and analyzing compiler warning defects. In *Proceedings of the 38th International Conference on Software Engineering* (Austin, Texas) (ICSE ’16). Association for Computing Machinery, New York, NY, USA, 203–213. <https://doi.org/10.1145/2884781.2884879>
- [32] Yoshiki Takashima, Chanhee Cho, Ruben Martins, Limin Jia, and Corina S. Păsăreanu. 2024. Crabtree: Rust API Test Synthesis Guided by Coverage and Type. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 293 (Oct. 2024), 30 pages. <https://doi.org/10.1145/3689733>
- [33] Yoshiki Takashima, Ruben Martins, Limin Jia, and Corina S. Păsăreanu. 2021. SyRust: automatic testing of Rust libraries with semantic-aware program synthesis. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (Virtual, Canada) (PLDI 2021). Association for Computing Machinery, New York, NY, USA, 899–913. <https://doi.org/10.1145/3453483.3454084>
- [34] Yixuan Tang, He Jiang, Zhide Zhou, Xiaochen Li, Zhilei Ren, and Weiqiang Kong. 2022. Detecting Compiler Warning Defects Via Diversity-Guided Program Mutation. *IEEE Transactions on Software Engineering* 48, 11 (2022), 4411–4432. <https://doi.org/10.1109/TSE.2021.3119186>
- [35] Rust Team. 2020. References and Borrowing - The Rust Programming Language. <https://doc.rust-lang.org/book/ch04-02-references-and-borrowing.html> [Online; accessed 2025-09-08].
- [36] Rust Team. 2020. Rust error codes index - Error codes index. https://doc.rust-lang.org/error_codes/error-index.html#rust-error-codes-index [Online; accessed 2025-09-10].
- [37] Rust Team. 2020. rust-lang/rust: Empowering everyone to build reliable and efficient software. <https://github.com/rust-lang/rust> [Online; accessed 2025-09-11].
- [38] TiKV. 2023. TiKV is a highly scalable, low latency, and easy to use key-value database. <https://tikv.org/>. (Accessed on 12/06/2023).
- [39] Justin Tracey and Ian Goldberg. 2023. Grading on a Curve: How Rust can Facilitate New Contributors while Decreasing Vulnerabilities. In *2023 IEEE Secure Development Conference (SecDev)*. 26–36. <https://doi.org/10.1109/SecDev56634.2023.00016>
- [40] Qian Wang and Ralf Jung. 2024. Rustlantis: Randomized Differential Testing of the Rust Compiler. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 340 (Oct. 2024), 27 pages. <https://doi.org/10.1145/3689780>
- [41] Wvyrn. 2025. Path remapping ICE from ‘unnecessary_transmutes’ suggestion · Issue #140277 · rust-lang/rust. <https://github.com/rust-lang/rust/issues/140277> [Online; accessed 2025-09-06].
- [42] Zhiwu Xu, Bohao Wu, Cheng Wen, Bin Zhang, Shengchao Qin, and Mengda He. 2024. RPG: Rust Library Fuzzing with Pool-based Fuzz Target Generation and Generic Support. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) (ICSE ’24). Association for Computing Machinery, New York, NY, USA, Article 124, 13 pages. <https://doi.org/10.1145/3597503.3639102>

- [43] Wenzhang Yang, Cuifeng Gao, Xiaoyuan Liu, Yuekang Li, and Yinxing Xue. 2024. Rust-twins: Automatic Rust Compiler Testing through Program Mutation and Dual Macros Generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA) (ASE '24). Association for Computing Machinery, New York, NY, USA, 631–642. <https://doi.org/10.1145/3691620.3695059>
- [44] Xizhe Yin, Yang Feng, Qingkai Shi, Zixi Liu, Hongwang Liu, and Baowen Xu. 2024. FRIES: Fuzzing Rust Library Interactions via Efficient Ecosystem-Guided Target Generation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) (ISSTA 2024). Association for Computing Machinery, New York, NY, USA, 1137–1148. <https://doi.org/10.1145/3650212.3680348>
- [45] Yuchen Zhang, Ashish Kundu, Georgios Portokalidis, and Jun Xu. 2023. On the Dual Nature of Necessity in Use of Rust Unsafe Code. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) (ESEC/FSE 2023). Association for Computing Machinery, New York, NY, USA, 2032–2037. <https://doi.org/10.1145/3611643.3613878>
- [46] Yehong Zhang, Jun Wu, and Hui Xu. 2025. RuMono: Fuzz Driver Synthesis for Rust Generic APIs. *ACM Trans. Softw. Eng. Methodol.* 34, 6, Article 169 (July 2025), 28 pages. <https://doi.org/10.1145/3709359>
- [47] Shuofei Zhu, Ziyi Zhang, Boqin Qin, Aiping Xiong, and Linhai Song. 2022. Learning and programming challenges of rust: a mixed-methods study. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania) (ICSE '22). Association for Computing Machinery, New York, NY, USA, 1269–1281. <https://doi.org/10.1145/3510003.3510164>

Received 2025-09-12; accepted 2025-12-22